Centro Universitário de Excelência
Sistemas de Informação

MecanIQA API — OAT 2. Desenvolvimento Web Orientado a Objetos

Autor: Arthur Machado Morais
Felipe Souza Teixeira da Silva
Ian da Silva Borges
Jonatan de Souza Ferreira
Victor Tadeu Araújo Augusto

Itabuna, 2026

Agenda

O objetivo desta apresentação é demonstrar a evolução da API REST da MecaniQA, aplicando conceitos avançados de Orientação a Objetos como DTOs, Padrão Builder e relacionamentos entre entidades para o gerenciamento completo de ordens de serviço.

1. Novas Entidades
Criação das classes OrdemServico, Pedido e ItemPedido para representar o fluxo de uma ordem de serviço completa.

2. DTOs
Implementação de objetos de transferência para separar a camada de modelo da camada de exposição da API.

3. Padrão Builder
Aplicação do padrão de projeto Builder para construção controlada e segura de Ordens de Serviço.

4. Controllers e API REST
Atualização dos controllers existentes e criação do OrdemServicoController com novos endpoints CRUD.

5.Validação e Entrega

Testes no Postman, diagramas UML (Classes e Atividade) e atualização do README e Collection.

Modelagem dos Dados

A modelagem foi expandida a partir das necessidades da MecaniQA, mantendo os princípios de Orientação a Objetos. Além das classes já existentes (Peça e Serviço), foram criadas três novas classes para representar o fluxo de uma ordem de serviço: OrdemServico, Pedido e ItemPedido.

A OrdemServico armazena id, descrição do problema, data de abertura, status e valor total. O Pedido contém id, descrição e valor. O ItemPedido é uma entidade associativa que armazena quantidade, preço unitário e referências para Peça e Serviço, permitindo que um pedido contenha múltiplos itens de diferentes tipos.

Novas Entidades do Model



Padrão Builder

O padrão de projeto Builder foi aplicado para permitir a construção controlada e segura de objetos da classe OrdemServico. Em vez de criar a OS diretamente com um construtor com muitos parâmetros, o Builder oferece uma interface fluente que facilita a leitura e evita erros.

A classe OrdemServicoBuilder possui métodos como comDescricao(), comStatus(), comValorTotal() e, por fim, build(), que retorna o objeto pronto. Isso torna o código mais limpo e expressivo, além de permitir a criação de objetos complexos passo a passo.

Como o Builder funciona



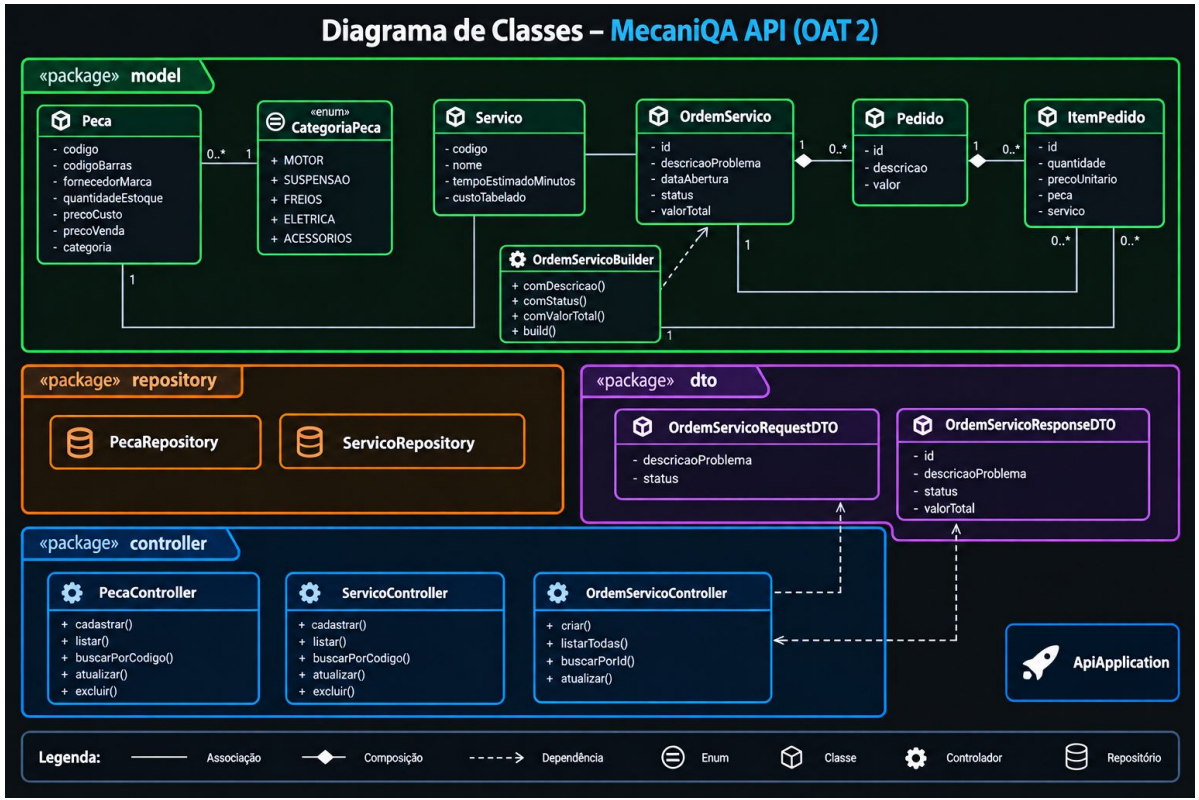
DTOs — Data Transfer Objects

Os DTOs (Data Transfer Objects) foram introduzidos para separar a camada de modelo da camada de exposição da API. Em vez de expor diretamente a entidade OrdemServico, criamos objetos específicos para requisições e respostas.

O OrdemServicoRequestDTO é usado para receber os dados do cliente (descrição do problema e status). Já o OrdemServicoResponseDTO é usado para devolver os dados da OS criada (id, descrição, status e valor total). Essa separação traz mais segurança, controle e flexibilidade à API.

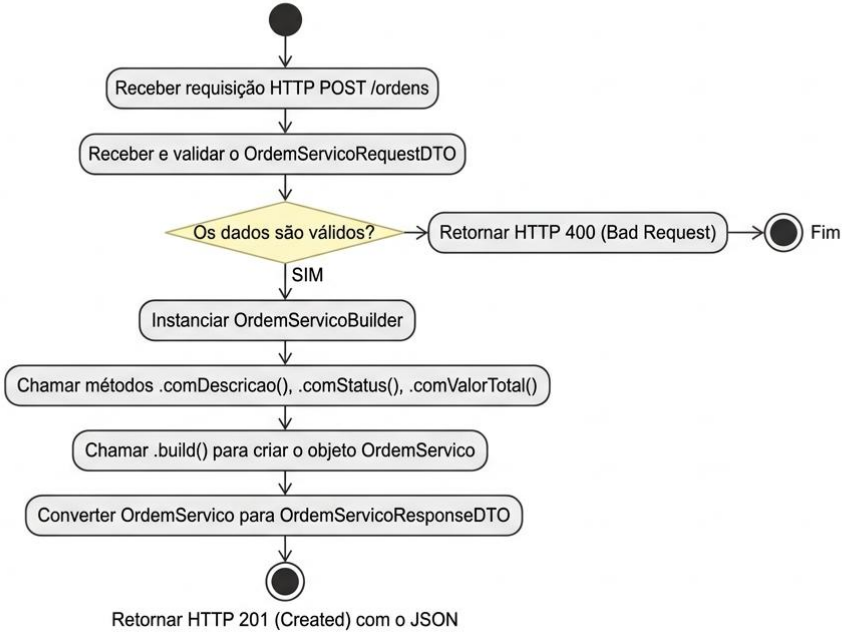
Request vs Response





MecanIQA API — OAT 2. Desenvolvimento Web Orientado a Objetos

Diagrama de Atividade – Endpoint POST /ordens

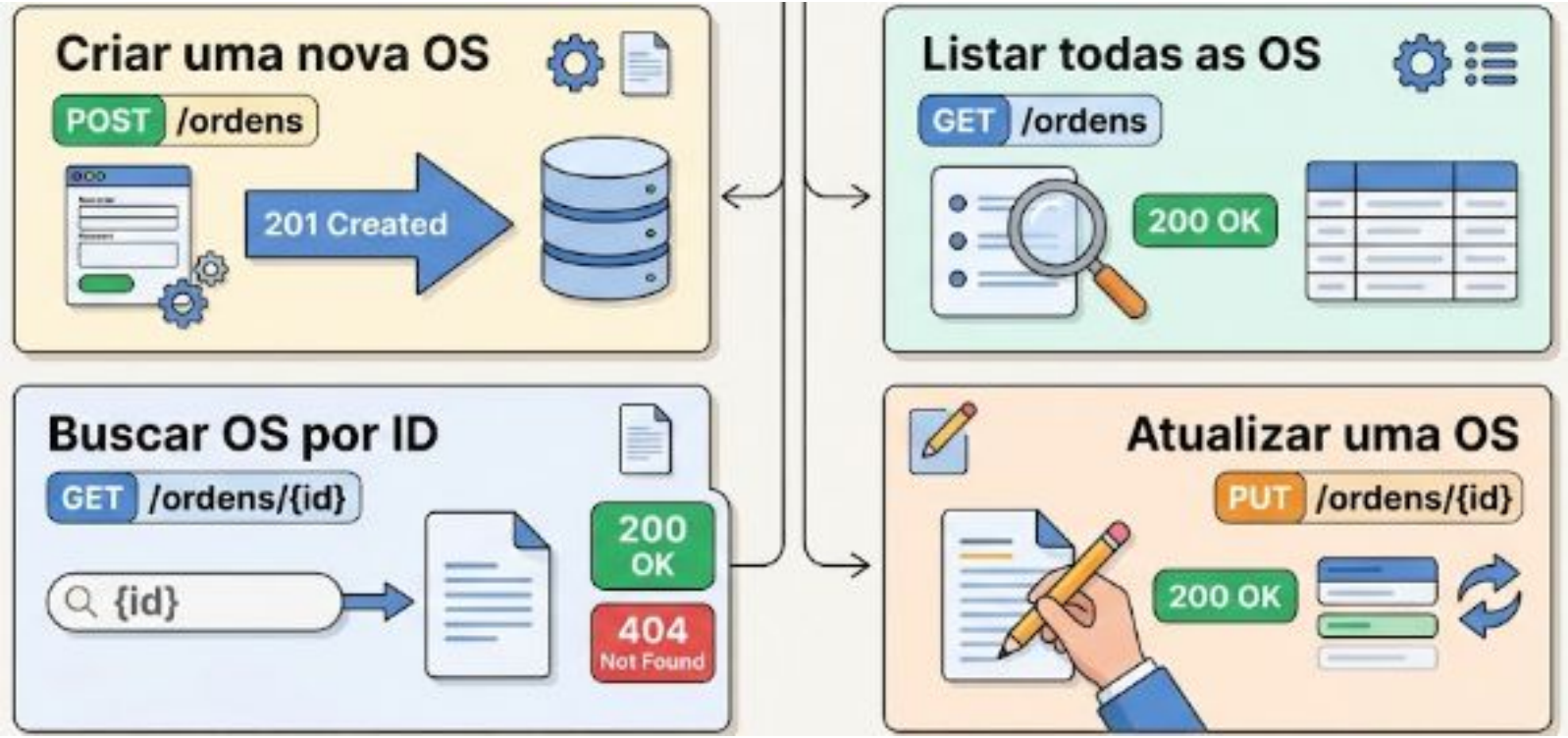


Controllers e API REST

Os Controllers continuam sendo responsáveis por receber as requisições HTTP e direcionar cada operação ao repository correspondente. Com a chegada da OAT 2, foi criado o OrdemServicoController, que implementa as operações de CRUD para as Ordens de Serviço.

A API agora conta com endpoints para criar, listar, buscar por ID e atualizar Ordens de Serviço. Cada operação retorna o status HTTP adequado, como 201 Created para criação, 200 OK para consultas e 404 Not Found para recursos inexistentes.

Novos Endpoints (CRUD da OS)



Validação e Entrega

A etapa de validação teve como objetivo verificar o funcionamento das novas operações implementadas. Os endpoints de Ordem de Serviço foram testados por meio do Postman, validando as operações de cadastro, consulta e atualização.

Também foram verificados os status HTTP retornados pela aplicação, contemplando cenários de sucesso e situações de erro. Como parte da entrega, o projeto foi complementado com o Diagrama de Classes atualizado, o Diagrama de Atividade e a Collection do Postman atualizada

Considerações finais

O desenvolvimento da MecaniQA API permitiu aplicar na prática conceitos avançados de Orientação a Objetos e desenvolvimento de APIs REST. A solução foi expandida com a modelagem de OrdemServico, Pedido e ItemPedido, além da aplicação do Padrão Builder e do uso de DTOs.

A separação entre Controllers, Repositories, Models e DTOs contribuiu para uma estrutura mais organizada, segura e de fácil manutenção. Os testes realizados permitiram validar o comportamento dos novos endpoints e os retornos HTTP da aplicação.

Referências

- https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods
- https://learning.postman.com/docs/
- https://refactoring.guru/pt-br/design-patterns/builder
- https://martinfowler.com/eaaCatalog/dataTransferObject.html
- Documentação Spring Boot